EcoNet + Weather Analytics

How to start the project and present it: start-up checklist, a 12–15 minute demo script, a backup plan and likely viva questions.

Part	What it covers	When
1. Starting everything	Docker, database views & trigger, Hadoop, browser tabs, PowerShell	15 min before
2. The presentation	Six sections with what to show and what to say	12–15 min
3. Backup plan	What to do if part of the live demo fails	Prepare a day before
4. Questions to expect	Short model answers for common viva questions	Revise beforehand

> **Golden rule:** do a full dry run a day before, starting from a freshly restarted laptop and following Part 1 exactly. Time yourself, and take screenshots of every successful step to use as backup.

Part 1 — Starting everything (do this 15 minutes before)

Run these in order. Each step depends on the one before it.

Step 1: Docker (database + app)

1. Open **Docker Desktop** and wait until it says "Engine running".

2. Open PowerShell and run:

```
docker stop rural-db
```

This frees port 5432. It is fine if it says the container is not running.

```
cd "C:\Users\KIIT0001\OneDrive\Desktop\econet\econet\econet-api"
```

```
docker compose up -d
```

3. Wait 30 seconds, then open **http://localhost:8081**.

Check: the dashboard loads.

> If the app will not load, run `docker compose down` and then `docker compose up -d`.

Step 2: Load the views and trigger (one time only)

The tables are created automatically, but the views and the trigger in `schema.sql` are not. Load them once so the DBMS features exist for the demo. Run from the `econet-api` folder:

```
Get-Content schema.sql -Raw | docker exec -i econet-db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
```

Check: you see `CREATE VIEW` and `CREATE TRIGGER` in the output.

Step 3: Hadoop (HDFS)

```
C:\hadoop\sbin\start-dfs.cmd
```

Leave the two windows it opens (NameNode and DataNode) running. Wait 20 seconds, then:

```
C:\java\jdk-11\bin\jps.exe
```

Check: you see **NameNode** and **DataNode** in the list. Then:

```
hdfs dfsadmin -safemode get
```

Check: `Safe mode is OFF`.

Step 4: Open these browser tabs, ready to go

#	URL	What it is
1	http://localhost:8081	Dashboard, open on the **Weather Analytics** tab
2	http://localhost:9870	Hadoop NameNode page
3	http://localhost:5050	pgAdmin, logged in and connected to the database
4	https://github.com/Bhavya-Rathor/Econet-	Your code on GitHub

Step 5: Open PowerShell, ready to go

```
cd "C:\Users\KIIT0001\OneDrive\Desktop\econet\econet\weather-analytics"
```

Leave it there. You will run the pipeline **live** during the demo.

Part 2 — The presentation (about 12–15 minutes)

#	Section	Time	Main thing to show
1	Problem & idea	1 min	Dashboard home page
2	Architecture	2 min	Architecture diagram
3	DBMS	3 min	pgAdmin: constraints, ROLLUP view, trigger
4	Live Hadoop run	4 min	PowerShell pipeline run + HDFS output
5	Forecast	2 min	Weather Analytics tab + evaluation file
6	Wrap-up	1 min	Summary

1. Problem & idea (1 min)

Show: the dashboard home page.

> **Say:** "EcoNet is a cloud-based environmental monitoring system for Vellore. It collects air, water, soil and weather data, stores it in PostgreSQL, processes it with Hadoop MapReduce, and forecasts the next three months. The project combines two subjects: DBMS for reliable storage, and cloud computing for distributed processing."

2. Architecture (2 min)

Show: draw this on the board or put it on a slide:

```
Open-Meteo API → PostgreSQL → HDFS → MapReduce → PostgreSQL → Dashboard
                      ↑                                ↓
               Spring Boot (REST API)          Forecast model
                      └──────── all in Docker ─────────┘
```

> **Say:** "Each part does what it is best at. PostgreSQL guarantees data integrity, Hadoop provides scalable processing, Spring Boot exposes a REST API, and Docker packages everything so it runs the same anywhere."

3. DBMS (3 min)

Show in pgAdmin: expand the tables.

- Open `weather_daily` → **Constraints**: show the **primary key**, the **foreign key to monitoring_zones** and the **UNIQUE (zone_id, obs_date)**.
- Run the ROLLUP view live:

```
SELECT * FROM v_weather_monthly_sql;
```

> **Say:** "The schema is normalised: raw data, computed results and predictions are in separate tables, linked to a zone by foreign keys. The unique constraint makes the pipeline idempotent, so re-running it updates rows instead of duplicating them. This view uses GROUP BY ROLLUP to compute the monthly averages and the grand total in one query, and I use it to validate the Hadoop output."

Bonus — the trigger demo (if time allows). Insert a dangerous reading and show that an alert appears automatically:

```
INSERT INTO environmental_readings (zone_id, domain, parameter_name, reading_value, unit, status, recorded_at)
VALUES (1, 'AIR', 'AQI', 250, 'AQI', 'NORMAL', NOW());

SELECT * FROM incident_alerts ORDER BY id DESC LIMIT 1;
```

> **Say:** "A PL/pgSQL trigger checks every insert against the threshold rules. AQI 250 is above the critical limit of 200, so the database itself created this alert. No application code was involved."

4. Live Hadoop run (4 min) — the highlight

Show: the PowerShell window. Run:

```
powershell -ExecutionPolicy Bypass -File .\run_pipeline.ps1 -Mode hadoop
```

Talk while it runs:

Stage	What to say
[1/5]	"Fetching 183 days of real weather from Open-Meteo into PostgreSQL."
[2/5]	"Exporting from the database into the format Hadoop reads."
[3/5]	"Uploading into HDFS and running the MapReduce job."

When it finishes, scroll up to the Counters and point:

```
Map output records=366          ← each day emitted twice (month + OVERALL)
Combine input records=366
Combine output records=7        ← Combiner squashed 366 → 7 before the shuffle
Reduce output records=7         ← 6 months + OVERALL
VALID_DAYS=183                  ← custom counter: every day parsed
```

> **Say:** "366 map outputs, because each day is emitted twice: once for its month and once for the overall total. The Combiner reduced 366 records to 7 before the shuffle, so far less data crosses the network. That is the main optimisation on a real cluster."

Explain the two design choices:

> **Say:** "The mapper emits sums, not averages, because an average of averages is wrong. Sums can be combined in any order. And wind direction uses a circular mean: averaging 350° and 10° normally gives 180°, south, but both winds are from the north. So I convert each angle to a vector."

Show: the verify table at the end (all `True`).

> **Say:** "Hadoop's output matches PostgreSQL's ROLLUP query exactly for all seven periods. I validated the distributed computation against the database."

Then switch to http://localhost:9870 → Utilities → Browse the file system → `/weather_output`:

> **Say:** "Here is the output stored in HDFS. _SUCCESS is Hadoop's completion marker."

5. Forecast (2 min)

Show: the dashboard's **Weather Analytics** tab, and open `forecast_evaluation.txt`.

> **Say:** "The first forecast used hard-coded rules and predicted rain would stop after September. Ten years of real data showed October and November are actually Vellore's wettest months, because of the northeast monsoon. So I replaced it with a seasonal harmonic regression that learns the yearly cycle from the data."

Point at the rainfall chart: "You can see the monsoon peak in October and November."

> **Say:** "I tested it fairly on five past seasons, each predicted using only earlier data. It is 31 to 83 percent more accurate on monthly averages. The one weakness is daily rainfall: no model can predict which exact day it will rain three months ahead."

> **Tip:** being honest about a limitation impresses examiners more than claiming perfection.

6. Wrap-up (1 min)

> **Say:** "To summarise: real data, a normalised database with integrity constraints, a verified MapReduce job on HDFS, a forecast that has been tested properly, and a dashboard. All of it runs in Docker and the code is on GitHub. The same design scales from 183 rows on a laptop to billions on a cluster without code changes."

Part 3 — Backup plan

Live demos can fail, so prepare these **beforehand**:

Risk	Backup
Hadoop fails live	Your screenshots: the job log with Counters, `/weather_output` in HDFS, and the verify table
Wi-Fi is down (Open-Meteo unreachable)	The backfill step fails, but the data is already in the database. Show the screenshots, or present the existing results in the dashboard.
Docker will not start	Screenshots of each dashboard tab, plus the code on GitHub
Examiner asks to see code	Open `WeatherMapper.java` and `WeatherReducer.java` and walk through them

> **A day before:** do a full dry run from a restarted laptop, following Part 1 exactly, and time yourself.

Part 4 — Questions to expect

1. Why use Hadoop for only 183 rows?

It demonstrates a technique that scales. The same code handles terabytes on a cluster.

2. What does the Combiner do, and when is it safe to use?

It pre-aggregates on the mapper side. It is only safe when the operation is associative and commutative, which is why the mapper emits sums.

3. What is normalisation?

Each fact is stored once. Example: zone details live only in `monitoring_zones`.

4. What happens if the pipeline runs twice?

The UNIQUE constraint plus the update logic means no duplicates (idempotent).

5. How do you know the Hadoop results are correct?

Independent validation against a SQL ROLLUP: all 7 periods match.

6. What is a trigger?

Code that runs inside the database automatically on INSERT, UPDATE or DELETE. Show the auto-alert demo.